## Project Identification

| PROJECT OVERVIEW | |
|---|---|
| Project Title | AI-Enhanced Network Anomaly Detection |
| Prototype Name | NOVA - Network Observation and Validation with AI |
| Purpose | Repeatable proof of concept that compares traditional rule-based, AI-based to detect network anomalies. |
| Team Members | Nira Nardeo, Jennifer Silva, Paul Mash and Kevin Villion |
| **SOURCE CONTROL & PROJECT STATUS** | |
| Repository | https://github.com/niranardeo/NOVA |
| Branch | feature/data-exploration |
| Evaluated Commit | f591ae0 |
| Project Timeline | https://github.com/users/niranardeo/projects/1/views/1 |

## Problem

Traditional intrusion detection systems can identify suspicious or malicious network traffic using methods such as signature-based, rule-based, or machine-learning detection. However, an alert alone may not provide enough information to explain why the activity was flagged, what may have caused it, or how an analyst should prioritize the investigation. This can make it difficult for cybersecurity analysts to quickly understand the significance of an alert and determine the appropriate response.

Another challenge is the number of false-positive alerts these systems may generate. Analysts may spend time investigating normal activity that has been incorrectly identified as suspicious, while more serious alerts are waiting to be reviewed. If an alert is incorrectly prioritized or dismissed as a false alarm, actual malicious activity could potentially go unnoticed. Improving the context provided with intrusion detection alerts could help analysts better understand suspicious network activity, reduce unnecessary investigations, and focus their attention on threats that present the greatest risk.

## Goals and Objectives:

1. Build a controlled proof-of-concept
2. Compare traditional and AI-supported detection.
3. Evaluate measurable detection performance.

## Research Question

To what extent can AI supplement traditional intrusion detection methods by improving the context and identification of likely causes of network traffic while minimizing false positives/false negatives?

## Scope

The project will develop and evaluate a proof of concept in a controlled VM environment using publicly available, labeled network datasets. The implementation focuses on establishing a machine-learning detection baseline and integrating it with a locally hosted AI component to provide additional context and help prioritize alerts. Suricata is being used as a rule-based comparison tool. The project is limited to offline testing and will not include live production monitoring, active scanning, penetration testing, automated blocking or mitigation, a full SOC/SIEM system, or the development of a new large language model.

The AI component will be used to help interpret alerts and identify possible contributing factors based on available evidence. However, a likely contributing factor will not be considered a confirmed root cause unless sufficient evidence supports that conclusion. The AI will be expected to acknowledge uncertainty rather than make conclusions that cannot be supported by the network data, and it will not dismiss an alert as benign without sufficient supporting evidence.

## Contribution

Network anomaly detection is an important cybersecurity problem because security analysts often must review large numbers of alerts and determine which ones present actual threats. Traditional intrusion detection methods can identify suspicious or unusual network traffic, but they may also generate false positives or provide limited context about why an event was flagged. This creates challenges for cybersecurity analysts and security operations teams to determine which alerts require immediate attention. Our project will evaluate whether AI can supplement traditional intrusion detection by providing additional context and helping analysts better understand and prioritize suspicious activity.

To evaluate this problem, the project will develop a Python-based proof of concept that processes labeled network data and uses Decision Tree and Random Forest machine-learning models for anomaly

detection. Model performance will be measured using accuracy, precision, recall, F1-score, false-positive rates, false-negative rates, and confusion matrices. Detection results will then be provided to a locally hosted AI model to evaluate whether it can provide useful context, identify possible contributing factors, and assist with alert prioritization. The expected contribution is a working prototype, and an evaluation of whether adding AI provides meaningful value beyond traditional detection methods. Rather than assuming that AI improves cybersecurity operations, the project will evaluate its usefulness, limitations, and potential to support analysts in making more informed decisions.

## Initial Research Foundation

1. **Binary Classification for Cyberattack Detection (foundational paper by Kevin Villon):** Our research builds upon a previous study [15] that used one of our selected datasets: WUSTL-IIoT-2021 [2]. The paper outlines methodologies for dataset preprocessing, including removing unnecessary attributes, reducing the risk of data leakage, and preparing training and testing datasets. It also evaluates the performance of three machine-learning classification algorithms: J48, IBk, and DecisionTable. This previous research provides a foundation for our approach to data preprocessing, classification, and performance evaluation in our proposed intrusion detection system.
2. **Cybersecurity Datasets**: Our research includes publicly available cybersecurity datasets such as CICIDS2017: Cleaned & preprocessed [1], WUSTL-IIoT-2021 [2], and UNSW-NB15 [3], which provide labeled network traffic for evaluating intrusion detection methods [6]. These datasets contain different types of network activity and attack scenarios, allowing us to compare detection performance across different environments. However, differences in dataset features, labeling, and traffic characteristics may affect model accuracy and the ability to compare results [7]. This research supports our approach to data preprocessing, feature selection, and validation.
3. **Threat Modeling and Security**: Our research also focuses on identifying security risks that could affect the reliability of our results. Microsoft's STRIDE [4] framework provides a structured approach to examining threats such as Tampering, Information Disclosure, and Denial of Service. NIST guidance on adversarial machine learning further highlights how manipulated data and inputs can affect AI system performance [8]. These sources support our decision to include data integrity checks, repository security controls, and testing potentially misleading AI inputs. They also reinforce the importance of ensuring that AI-generated explanations do not override the original detection results [9].
4. **Detection Methods and AI Tools**: Our research examines traditional intrusion detection methods, machine-learning classifiers, and AI-assisted alert interpretation. Suricata provides a rule-based detection approach, while Python and scikit-learn support the development and evaluation of Decision Tree and Random Forest models [10]. Previous research has demonstrated that these machine-learning methods can effectively identify certain types of

malicious network traffic, although their performance depends on the datasets and testing methods used [5] [11]. We also reviewed research on AI-assisted security analysis to understand whether locally hosted AI can provide meaningful context and help prioritize alerts [12]. These findings will guide our comparison of detection performance, AI-generated explanations, and the limitations of combining these approaches.

### *Methodology Overview and Architecture*

The NOVA project uses a controlled proof-of-concept approach to evaluate whether AI can improve network anomaly alert interpretation and prioritization while maintaining human oversight. The architecture includes three components: traditional rule-based detection using Suricata and planned threshold rules [13], Decision Tree and Random Forest machine-learning classifiers [6], [7], and a planned AI-assisted analysis layer [14]. The machine-learning classifiers serve as detection baselines rather than traditional intrusion detection systems. The AI component will provide additional context and prioritization without changing the original detection results.

### *Data Processing and Model Development*

The team verified a cleaned dataset 'CICIDS2017: Cleaned & preprocessed, [1] ' containing 2,520,751 network traffic records, 52 numerical features, and seven traffic categories. A stratified 70/15/15 split was established for training, validation, and testing using a random seed of 42. The validation partition contains 378,113 records, while the test partition remains reserved for final evaluation.

Two classifiers were developed using scikit-learn. The Decision Tree was configured with a maximum depth of 15 and a minimum of 10 samples per leaf to limit model complexity. The Random Forest used 50 trees with the same depth and leaf-size settings, allowing comparison between a single decision tree and an ensemble model. Both models used 176,452 training records, 52 input features, and a random seed of 42. The Random Forest was configured to use two parallel jobs.

The practical component includes model-development notebooks and a standalone Python runner for repeatable training and evaluation. The workflow follows this sequence:

**Prepared CICIDS2017: Cleaned & preprocessed Data → Training and Validation → Decision Tree / Random Forest → Seven-Class Predictions → Performance Metrics → Error Analysis → AI-Assisted Interpretation (Planned)**

The runner also performs input validation and optional duplicate/overlap auditing. Predictions are mapped into normal-versus-attack categories to distinguish between false positives, false negatives, and attack-to-attack misclassifications. Structured error reports will provide consistent inputs for the planned AI-assisted analysis.

## Experiment Design and Next Steps

The project uses a local Python environment with Jupyter Notebooks, pandas, NumPy, and scikit-learn. Initial model development and validation have been completed, and the standalone runner has passed automated tests and an end-to-end smoke test. The full-dataset runner experiment remains pending while the team reviews data-quality findings and aligns model configurations.

The next phase will compare machine-learning results with Suricata and threshold-based rules, followed by AI-assisted analysis of structured alerts. The team will evaluate whether AI provides accurate, evidence-supported explanations, identify possible contributing factors, and improve prioritization without overriding detector outputs.

## Threat Model and Security Requirements:

The team selected the STRIDE threat-modeling framework to identify risks affecting the accuracy, integrity, and reliability of project results [4]. Key assets include datasets, Python scripts, machine-learning models, configurations, detection results, and AI explanations. Potential threats include unauthorized access, malicious data or dependencies, and human error. The main trust boundaries exist between external datasets and the detection models, the detector and AI component, and AI explanations and human analysts.

The primary STRIDE threats are Tampering, Denial of Service, and Information Disclosure. To address tampering, the team will validate data quality, verify file integrity using SHA-256 hashes, and track changes through GitHub. DoS risks will be evaluated through controlled workload testing to assess system stability. To prevent information disclosure, the team will review its public GitHub repository and use secret-scanning tools to identify exposed credentials or sensitive information. [16]

Additional risks include incorrect classifications and misleading AI explanations. The team will evaluate model performance using unseen test data and measure false-positive and false-negative rates. Structured alerts will help ensure AI-generated interpretations do not alter original detection results. Testing will include simulated malicious instructions to verify this separation, while restricted permissions and experiment logs will support accountability. These controls provide measurable ways to assess the project's security and reliability.

## Environment, Data, Ethics, and Authorization

| Tool / Component | Version / Status | Purpose |
|---|---|---|
| Python | 3.11 / 3.13.7 | Data processing, scripting, and model development |
| Jupyter Notebook | 7.6.3 / Version not specified | Dataset exploration, preprocessing, model development, and evaluation |
| Pandas | 3.0.3 in documented environment | Loading, inspect and organize datasets |
| NumPy | 2.4.6 in documented environment | Numerical processing and data preparation |
| Scikit-learn | 1.9.0 in documented environment | Decision Tree, Random Forest, and performance evaluation |
| Matplotlib | Version to be confirmed | Visualization of model results |
| Unittest | Python standard library | Automated testing of Python scripts |
| Git and GitHub | In use | Version control, team collaboration, and documentation |
| CICIDS2017: Cleaned & preprocessed | Public dataset, cleaned and processed | Network intrusion-detection testing and evaluation |
| Suricata | Planned | Rule-based detection comparison |
| Local AI model | Planned | Alert interpretation, contextualization, and prioritization |

The team uses a publicly available, cleaned dataset CICIDS2017: Cleaned & preprocessed containing labeled normal and attack traffic. All experiments are conducted offline using local dataset copies. No live traffic collection, unauthorized scanning, automated blocking, or production deployment is included. Dataset files and generated outputs are excluded from the public repository, while scripts and documentation are maintained in GitHub. The team will follow applicable dataset licensing requirements and review published materials for sensitive information or exposed credentials.

Data-quality auditing identified 190 feature vectors shared between the training and validation partitions, including 32 with matching labels and 158 with conflicting labels. These findings raise concerns about data independence and label reliability. The team will document these limitations and determine whether preprocessing adjustments are necessary before final evaluation. AI-generated explanations will remain advisory and subject to human review.

### *Evaluation Approach and Preliminary Results*

The team evaluates model performance using accuracy, precision, recall, macro and weighted F1-scores, confusion matrices, false-positive and false-negative rates, and training time. Macro F1-score and class-specific recall are emphasized because the dataset contains substantially more normal traffic than certain attack categories.

Both models were evaluated against the same 378,113 validation records, producing the following preliminary results:

| **Evaluation Metric** | **Decision Tree** | **Random Forest** |
|---|---|---|
| Multiclass accuracy | 99.71% | 99.75% |
| Macro F1-score | 85.68% | 91.22% |
| Weighted F1-score | 99.68% | 99.74% |
| Training time | 3.58 seconds | 6.94 seconds |
| True positives | 63,137 | 63,136 |
| True negatives | 313,931 | 314,053 |
| False positives | 328 | 206 |
| False negatives | 717 | 718 |
| False-positive rate | 0.1044% | 0.0656% |
| False-negative rate | 1.1229% | 1.1244% |

Random Forest achieved stronger overall multiclass performance, improving macro F1-score by 5.54 percentage points and reducing false positives by 122 records. However, Decision Tree missed one fewer attack. Both models struggled with less common attack categories, particularly Bot traffic, where Random Forest improved recall from 6.85% to 35.62% but still incorrectly classified most Bot records. Random Forest also showed a slightly lower recall for Brute Force and Web Attacks.

These findings demonstrate why overall accuracy alone is insufficient for evaluating cybersecurity models. False positives represent normal traffic incorrectly flagged as malicious, while false negatives represent attacks incorrectly classified as normal. Attack-to-attack classification errors are evaluated separately because they do not necessarily represent missed attack detection.

### *Planned Evaluation*

The next phase will compare machine-learning results with traditional rule-based detection using Suricata and planned threshold rules. The team will then evaluate whether a locally hosted AI component improves alert interpretation and prioritization. Success will be measured by the accuracy and usefulness of AI explanations, consistency with available evidence, and processing time. Failure

cases will include missed attacks, false alerts, incorrect classifications, and unsupported AI explanations.

Security validation will focus on protecting dataset and model integrity, maintaining the separation between detection results and AI interpretations, and preventing sensitive information from entering the public repository. Tests will include file-integrity checks, controlled workload testing, and simulated malicious instructions to verify that AI cannot modify original classifications.

### *Evidence and Reproducibility*

The team will preserve Python scripts, Jupyter Notebooks, model configurations, preprocessing steps, performance metrics, confusion matrices, classification-error reports, and automated test results. Planned evidence will include Suricata comparisons, AI-generated explanations, security-validation findings, and final test results.

The NOVA GitHub repository will document software dependencies, dataset preparation, model parameters, execution procedures, and random seeds so that another qualified person can reproduce and evaluate the experiments. Final evaluation will use the reserved test partition to assess whether preliminary findings remain consistent on previously withheld data.

### *Working Agreement, Roles, and Team Coordination*

| TEAM MEMBER | PRIMARY ROLE | PRIMARY RESPONSIBILITIES | BACKUP / SUPPORT |
|---|---|---|---|
| **NIRA NARDEO** | Project Manager | Documentation, timeline management, project coordination | Testing, research, presentation |
| **JENNIFER SILVA** | Technical Lead | Python development, model development, integration | Research, testing, documentation |
| **KEVIN VILLON** | Data & Quality Assurance | Dataset preparation, metrics, validation, video | Python scripting, research, testing, documentation |
| **PAUL MASH** | Programming | Python development, research, analysis | Documentation, testing |

The team meets weekly through Discord to review progress, discuss technical challenges, and coordinate upcoming activities. The team is constantly connected via group messaging and email as well. GitHub serves as the central repository for code, notebooks, documentation, and version control, while GitHub Projects is used to manage task assignments, internal deadlines, deliverables, and progress. The GitHub Projects board provides the current project schedule, including assigned responsibilities, dates and task status.

Team members are responsible for maintaining their assigned tasks, communicating blockers, and reviewing shared deliverables. Technical decisions and disagreements are addressed through team discussions and consensus. GitHub commits, pull requests, and task updates provide visibility into contributions and support accountability throughout the project.

## References:

[1] E. A. C. Ribeiro, “CICIDS2017: Cleaned & preprocessed,” Kaggle. [Online]. Available: https://www.kaggle.com/datasets/ericanacletoribeiro/cicids2017-cleaned-and-preprocessed

[2] M. Zolanvari, M. A. Teixeira, L. Gupta, K. M. Khan, and R. Jain. "WUSTL-IIOT-2021 Dataset for IIoT Cybersecurity Research," Washington University in St. Louis, USA, October 2021, http://www.cse.wustl.edu/~jain/iiot2/index.html

[3] University of New South Wales, “The UNSW-NB15 Dataset,” UNSW Research. [Online]. Available: https://research.unsw.edu.au/projects/unsw-nb15-dataset

[4] Microsoft. (2022, August 25). *Microsoft Threat Modeling Tool threats*. Microsoft Learn. https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats

[5] Rodríguez M, Alesanco Á, Mehavilla L, García J. Evaluation of Machine Learning Techniques for Traffic Flow-Based Intrusion Detection. Sensors (Basel). 2022 Nov 30;22(23):9326. doi: 10.3390/s22239326. PMID: 36502028; PMCID: PMC9740321.

[6] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, Funchal, Madeira, Portugal, Jan. 2018, pp. 108–116. [Online]. Available: https://doi.org/10.5220/0006639801080116

[7] G. Engelen, V. Rimmer, and W. Joosen, “Troubleshooting an intrusion detection dataset: The CICIDS2017 case study,” in *Proc. IEEE Secur. Privacy Workshops (SPW)*, 2021, pp. 7–12. [Online]. Available: https://doi.org/10.1109/SPW53761.2021.00009

[8] A. Vassilev, A. Oprea, A. Fordyce, H. Anderson, X. Davies, and M. Hamin, “Adversarial machine learning: A taxonomy and terminology of attacks and mitigations,” National Institute of Standards and Technology, Gaithersburg, MD, USA, NIST AI 100-2e2025, Mar. 2025. [Online]. Available: https://doi.org/10.6028/NIST.AI.100-2e2025

[9] E. Tabassi, “Artificial intelligence risk management framework (AI RMF 1.0),” National Institute of Standards and Technology, Gaithersburg, MD, USA, NIST AI 100-1, Jan. 2023. [Online]. Available: https://doi.org/10.6028/NIST.AI.100-1

[10] Open Information Security Foundation, “Suricata documentation.” Accessed: Oct. 12, 2026. [Online]. Available: https://docs.suricata.io/

[11] A. L. Buczak and E. Guven, “A survey of data mining and machine learning methods for cyber

security intrusion detection," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 2, pp. 1153–1176, 2nd Quart. 2016. [Online]. Available: https://doi.org/10.1109/COMST.2015.2494502

[12] M. Rieger, A. Shah, A. Alam, and M. J. Hossain, "Possibilities and limitations of using large language models (LLMs) for alert classification and prioritisation in security operations centers (SOCs)," *Expert Syst. Appl.*, vol. 331, Art. no. 133194, 2026. [Online]. Available: https://doi.org/10.1016/j.eswa.2026.133194

[13] A. Schärmer, M. Landauer, F. Skopik, M. Wurzenberger, and M. Squarcina, "Let the alerts speak: LLM-based IDS alert interpretation for SOC triage," in *Availability, Reliability and Security: ARES 2026 Int. Workshops, Part II*, ser. Lecture Notes in Computer Science, vol. 16901, Springer, 2026, pp. 346–364. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-032-35579-9_19

[14] B. A. Alahmadi, L. Axon, and I. Martinovic, "99% false positives: A qualitative study of SOC analysts' perspectives on security alarms," in *Proc. 31st USENIX Secur. Symp.*, Boston, MA, USA, Aug. 2022, pp. 2783–2800. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/alahmadi

[15] K. Villon, "Binary Classification for Cyberattack Detection," unpublished course project, CS 619, Seidenberg School of Computer Science and Information Systems, Pace University, May 14, 2026. https://github.com/niranardeo/NOVA/blob/main/Binary%20Classification%20for%20Cyberattack%20Detection%20-%20Kevin%20Villon.pdf

[16] GitHub. (n.d.). *Secret scanning*. GitHub Docs. GitHub. (n.d.). *Secret scanning*. GitHub Docs. https://docs.github.com/en/code-security/concepts/secret-security/secret-scanning?learn=secret_scanning

[17] Scikit-learn Developers, "DecisionTreeClassifier," *Scikit-learn Documentation*. https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html

[18] Scikit-learn Developers, "RandomForestClassifier," *Scikit-learn Documentation*. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

[19] Scikit-learn Developers, "Metrics and Scoring: Quantifying the Quality of Predictions," *Scikit-learn Documentation*. https://scikit-learn.org/stable/modules/model_evaluation.html

[20] Project Jupyter, "Project Jupyter Documentation." https://docs.jupyter.org/

[21] OpenAI, "ChatGPT," AI-assisted development and documentation tool. https://chatgpt.com/

[22] Abiramasundari, S., & Ramaswamy, V. (2025). Distributed denial-of-service (DDOS) attack detection using supervised machine learning algorithms. *Scientific Reports, 15*, 13098. https://doi.org/10.1038/s41598-024-84879-y

AI used; see appendix in github repository titled: IT 691 Capstone Fall 2026 Team 1 NOVA AI Appendix.xlsx